

Simplify Database Migration by Using Object Relational Mapping

Web applications are developed using different programming languages for various components. Effecting changes to the database applications within the app is a cumbersome process and is time consuming. Object relational mapping is a lazy person's way out, making it simple to migrate data.



ORM or object relational mapping is a technique or a toolkit to access the database from programming languages or frameworks. Though ORM has been used in the industry for a long time, many people are not too aware about it. This article throws light on this topic, with budding developers as its target audience.

The normal scenario of a project

Let's look at a Web application. The front-end/Web pages might be programmed in HTML/ jQuery and the server side may be in any scripting language like PHP or a framework like Django. The database may be Oracle or SQL Server.

The connection to the database is coded (called the connection string) in the scripting language, specific to the database, by giving the following parameters:

`dbname, user, password, port number, etc.`

Inside the application, we might have written several SQL queries that are specific to the vendor database.

Disadvantages

Let's look at a scenario in which, for certain business purposes, the client wants to change to some open source database like MySQL. It is a difficult/ cumbersome activity for the developers involved to change all the DB queries to MySQL, including the connection string. This would mainly affect the time required to do the development/changes and the testing – both of which might result in a lower profit margin.

How ORM fits into the picture

Tables in the database schema are defined as classes in Django, the widely used and the most popular framework of Python.

An example of an employee class/table is:

```
from django.db import models
class Employee(models.Model):
    first_name = models.CharField(max_length=30)
    last_name = models.CharField(max_length=30)
    age = models.IntegerField()
```